



L02: Basic Python Constructs

Dr. Todd Jones

t.r.jones@reading.ac.uk

Department of Computer Science

Continuing on from the introduction to Python variables and your new familiarity with Jupyter Notebooks...

Lecture Objectives

- Differentiate, interact with, implement, and modify **data types** available in Python.
 - Identify, configure, and apply **flow control** structures and **comparison** operators.
 - Understand and implement the **comprehension** construct and the "**walrus operator**".
-

Data Types

Variables can store data of different **types**, each with different properties.

Default types in Python:

Type Category	Type Representation
Text	str
Numeric	int, float, complex
Sequence	list, tuple, range
Mapping	dict, {frozendict}
Sets	set, frozenset
Boolean	bool
Binary	bytes, bytearray, memoryview
None	NoneType

Data Type Examples

```
In [ ]: # Text -----  
# Strings  
fstname = "Mike"
```

```
In [ ]: # Numeric -----  
x = 1          # Integer  
y = 2.5        # Floating point  
z = 3 + 2.1j    # Complex
```

```
In [ ]: # Sequence -----  
a = [2, 3, 2, 3, 3] # List  
t = (1, 2, 3)       # Tuple  
r = range(6)        # Range
```

```
In [ ]: # Mapping -----  
d = { 1: "Hello", 2: "world" } # Dictionary
```

```
In [ ]: # Sets -----  
s = {'A', 'B', 'A', 'C', 'A'} # Set  
f = frozenset({'A', 'C', 'A'}) # Frozen set
```

```
In [ ]: # Boolean -----  
yes = True  
no = False
```

```
In [ ]: # Binary -----  
b = bytes(7)          # byte  
ba = bytearray(7)     # bytearray  
m = memoryview(bytes(5)) # memoryview
```

```
In [ ]: # None -----  
n = None # None
```

```
In [ ]: %whos
```

Numbers

- `int`, or integer, is a whole number, positive or negative, without decimals, of **unlimited length**.
- `float`, or floating point number, is a number, positive or negative, containing one or more decimals.
 - `float` can also hold numbers in scientific notation, with an "e" or "E" to indicate the power of 10.
 - 64-bit

```
In [ ]: # integers  
n1 = 1  
n2 = 35656222554887711  
n3 = -3_255_522  
  
print(type(n1))  
print(type(n2))  
print(type(n3))
```

```
In [ ]: # floats
f1 = 1.100_543
f2 = 1.0
f3 = -35.59
f4 = 5_135e3
f5 = 12E4
f6 = -87.7e100
```

```
print(type(f1))
print(type(f2))
print(type(f3))
print(type(f4))
print(type(f5))
print(type(f6))
```

```
In [ ]: f1
```

```
In [ ]: # Report largest floating point value
```

```
import sys
sys.float_info.max
```

```
In [ ]: from math import factorial
```

```
huge = factorial(500)

print(huge)
print(type(huge))
print(huge.bit_length())
```

```
# Integers are variable size, stored in base 2**30
```

Arithmetic Operations

- **Arithmetic operators** are used with numeric values to perform common mathematical operations.
- Python follows mathematical conventions when evaluating mathematical expressions:
 - expressions in **parentheses** are performed first,
 - then, **exponentiation** (*right-to-left*),
 - next is **multiplication/division/remainder**,
 - and then **addition/subtraction**,
 - from **left to right**.

```
In [ ]: # Arithmetic operators
```

```
x = 10
y = 6

print('Addition:      {} + {} = {}'.format(x, y, x + y))
print('Subtraction:   {} - {} = {}'.format(x, y, x - y))
print('Multiplication: {} * {} = {}'.format(x, y, x * y))
print('Exponentiation: {} ** {} = {}'.format(x, y, x**y))
print('Division:       {} / {} = {}'.format(x, y, x / y))
print('Floor Division: {} // {} = {}'.format(x, y, x // y))
print('Modulus:        {} % {} = {}'.format(x, y, x % y))
```

```
In [ ]: # Multiplication before addition
```

```
print( 16 + 2 * 11)
print((16 + 2) * 11)
```

```
In [ ]: # Exponentiation from the right
```

```
print( 2 ** 3 ** 2)
print((2 ** 3) ** 2)
```

```
In [ ]: # Equal-precedence operations from the left
```

```
print( 20 / 4 * 2)
print( (20 / 4) * 2)
print( 20 / (4 * 2))
```

Booleans

- **Booleans** represent one of two values: `True` or `False`.
- When you **compare** two values, the expression is evaluated and Python returns the Boolean answer.
- **Comparison operators** are used to compare two values.
- **Logical operators** are used to combine Booleans.

- `not`
- `and`
- `or`

```
In [ ]: # Comparison operators
```

```
x = 5
y = 10

print('Equal:           {} == {} is {}'.format(x, y, x == y))
print('Not equal:       {} != {} is {}'.format(x, y, x != y))
print('Greater than:    {} > {} is {}'.format(x, y, x > y))
print('Greater than or equal to: {} >= {} is {}'.format(x, y, x >= y))
print('Less than:       {} < {} is {}'.format(x, y, x < y))
print('Less than or equal to: {} <= {} is {}'.format(x, y, x <= y))
```

```
In [ ]: # Logical operators
```

```
x = 5
y = -10

print('not: not({} > 0) is {}'.format(x, not (x > 0)))
print('and: {} > 0 and {} > 0 is {}'.format(x, y, (x > 0 and y > 0)))
print('or:  {} > 0 or {} > 0 is {}'.format(x, y, (x > 0 or y > 0)))
```

Logical Operators

`not`

x	not x
---	-------

False	True
-------	------

x	not x
True	False

and

x	y	x and y
False	False	False
False	True	False
True	False	False
True	True	True

or

x	y	x or y
False	False	False
False	True	True
True	False	True
True	True	True

```
In [ ]: False == 0
```

```
In [ ]: True == 1
```

```
In [ ]: # Identity check: `is`
# if the two variables point to the exact same object
False is 0
```

Bitwise Operators

While we might use some of these when comparing `True` and `False` values, and they can be useful in a number of different numerical manipulations of binary values.

When you have a need, see: <https://realpython.com/python-bitwise-operators>.

Note: There is no sign bit in Python (it is part of the object's metadata). Integers can have an infinite number of bits.

Operator	Example	Meaning
&	a & b	Bitwise AND
	a b	Bitwise OR
^	a ^ b	Bitwise XOR (exclusive OR)
~	~a	Bitwise NOT
<<	a << n	Bitwise left shift
>>	a >> n	Bitwise right shift

```
In [ ]: # Convert an ASCII character to 8-bit binary
char = 'H'
print(ord(char), '# Unicode')
print(f"{ord(char):08b}")
```

```
In [ ]: # Similarly, "\0b"-prefixed string representation
bin(72)
```

```
In [ ]: # Shift left by 1, (multiply by 2)
bin(72 << 1)
```

```
In [ ]: # Cast binary string back to integer
int(bin(72 << 1), 2)
```

Strings

- A **string** is a sequence of characters.
- Define a string using single quotations `' '` or double quotations `" "`.
- The `+` operator **concatenates** strings.
- The `*` operator **replicates** strings.
- The `len()` function returns the length of a string.

```
In [ ]: # Basic string usage

greeting = 'Hello'
target = "everyone"
intro = "I'm Fiona." # use "" to include a ' in a string
print(greeting, target)

print(intro)
```

```
In [ ]: greeting + target
```

```
In [ ]: greeting * 3
```

```
In [ ]: len(greeting)
```

Indexing

- Characters in a string can be accessed using **zero-based indexing** using **square bracket** notation: `variable[n]`.
 - Think of the string like it is an array of some sort.
- The `len()` function returns the length of a string.
- For a string `s` with `x = len(s)`, the valid indices range from $0, 1, \dots, x - 1$.
- Attempting to index a string outside of its bounds will produce a runtime error.
- A **negative** index begins counting from the other end, starting from -1 .

s =	"a	b	c	d	e	f	g"
+ve index	0	1	2	3	4	5	6
character	a	b	c	d	e	f	g
-ve index	-7	-6	-5	-4	-3	-2	-1

```
In [ ]: s = 'abcdefg'

s[1]
```

```
In [ ]: s[-2]
```

Slicing

- Indexing as: `[start_index : end_index : step]` returns a slice of a string.
- The `start_index` is **included**.
- The `end_index` is **excluded**.

```
In [ ]: # From position 2 to 5 (not including 5)
# s = 'abcdefg'
s[2:5]
```

```
In [ ]: # From position 1 to 5 (not including 5), by 2
# s = 'abcdefg'
s[1:5:2]
```

```
In [ ]: # Slice from the start to position 3 (not including 3)
# s = 'abcdefg'
s[:3]
```

```
In [ ]: # Slice from position 4 to the end
# s = 'abcdefg'
s[4:]
```

```
In [ ]: # Slice from position -5 to the end
# s = 'abcdefg'
s[-5:]
```

```
In [ ]: # Entire slice
# s = 'abcdefg'
s[:]
```

```
In [ ]: # Entire slice, by 2
# s = 'abcdefg'
# ???
```

```
In [ ]: # Entire slice, reverse order
# s = 'abcdefg'
# ???
```

NOTE: Strings are **immutable**.

- Their individual elements **cannot** be changed!

```
In [ ]: # Trying to change an individual character of a string will yield an error:
# s = 'abcdefg'
s[0] = 'A'
```

There are many useful string manipulation methods:

https://www.w3schools.com/python/python_ref_string.asp

Lists

Lists are *collections* of objects, having the following properties:

- Allow duplicate members
- Can store multiple types
- Can be nested
- Ordered
- **Mutable**
- Can have variable length

In other words, objects are stored in the list in an order, and the list can be modified.

Lists can be made longer or shorter, and can store combinations of different types of object in a single list, **including other lists**.

Lists are implemented as **dynamic arrays**, arrays of pointers to Python objects, so they can be dynamically resized as needed.

- Define a list by writing comma separated values in square brackets `[]`.
- The `len()` function returns the length of a list.

```
In [ ]: list1 = [] # creates an empty list
list2 = [1, 2, 3, 2, 1]
list3 = [True, 3.14, 2, "one"]
list4 = [[1, 2, 3], 2, "seven"]

print('list1 length:', len(list1))
print('list2 length:', len(list2))
print('list3 length:', len(list3))
print('list4 length:', len(list4))
```

```
In [ ]: list3[-1]
```

```
In [ ]: list4[0]
```

```
In [ ]: list4[0][1]
```

```
In [ ]: list4[2][2]
```

List operators and properties

- `+` concatenates lists.

- `*` duplicates lists.
- Lists are **mutable**.
 - Individual entries can be modified after a list is created.

```
In [ ]: # Concatenate two lists
```

```
a = [1, 2, 3]
b = ['m', 'n']

c = a + b
c
```

```
In [ ]: # Repeat a List
```

```
c * 3
```

```
In [ ]: # Replace an entry in the list
```

```
c[0] = 7012
c
```

```
In [ ]: # Perform a membership test using the `in` operator
```

```
7012 in c
```

Some List Methods

Lists in Python are objects that have **methods**. For any list, `a`, we can apply:

- `a.append(x)`
 - append value `x` to list `a`.
- `a.insert(i, x)`
 - insert value `x` into list `a` at index `i`.
- `a.remove(x)`
 - remove the first matching instance of `x`.
- `a.sort()`
 - sort list `a` in ascending order.
- `a.reverse()`
 - reverse list `a`.
- `a.extend(s)`
 - adds all elements in *iterable* object `s` to the end of the list.

Note that these modify the list object **in place**, so there is no need to write `a = a.append(x)`.

- In fact, doing so will set `a` to the return value of the append method; here the special value `None` representing a null object!

More: https://www.w3schools.com/python/python_ref_list.asp

```
In [ ]: a = [1, 4, 2, 8, 5, 0, 9, 0]
        a.append(200)
        a
```

```
In [ ]: a.insert(2, 500)
        a
```

```
In [ ]: a.remove(0)
        a
```

```
In [ ]: a.sort()
        a
```

```
In [ ]: b = ['a', 'b', 'd', 'w', 'c']
        b.sort()
        b
```

```
In [ ]: b.reverse()
        b
```

```
In [ ]: b.extend('777')
        b
```

```
In [ ]: b = b.extend('888')
        b
```

Tuples

Tuples are *collections* of objects, similar to lists:

- Allow duplicate members
- Can store multiple types
- Can be nested
- Ordered

But unlike Lists, Tuples are

- **Immutable**

Define a tuple by writing comma separated values in **parentheses** `()`.

Indexing, slicing, `len()`, `+` (concatenation), `*` (repetition), `in` (membership test) work in the same way as strings and lists.

Additionally, `.count( value )` returns the number of times a specified value occurs.

```
In [ ]: # Create a tuple
        t = (1, 1, 'two', 3.14, [4, 5, 6], 'abc')

        len(t)
```

```
In [ ]: 'three' in t
```

```
In [ ]: t.count(1)
```

```
In [ ]: # Trying to change an individual item of a tuple will yield an error:
```

Dictionaries

Dictionaries act as efficient lookup tables of **key:item** pairs. They are:

- **Order**
 - As of Python version **3.7**, dictionaries are **ordered**.
 - In Python **3.6** and earlier, dictionaries are unordered.
 - No *positional index* in either case, though.
 - Useful property for *iteration*.
 - Operations like concatenation, repetition, slicing are not available in either case.
- Entries are accessed by **keys**.
 - No duplicated members!
 - Each **key** must be unique.
- **Mutable**
- Nestable
- Dynamically sized

Define a **dictionary** by a pair of curly braces `{}` with `key:value` pairs separated by commas.

- Each key has an associated value.
- Keys or values do not all have to have the same type.
- Keys can only be immutable types:
 - integers
 - strings
 - tuples
 - frozensets

```
In [ ]: # An empty dictionary can be created using {}. Values are then
        # assigned to keys using a syntax similar to lists:
```

```
d = {}

d['Python'] = 'Snake' # Key 'Python' has value "Snake"

d[4] = 7              # Key 4 has value 7

d
```

```
In [ ]: # Once a dictionary has been created, values corresponding to a
        # key can be accessed:
```

```
d['Python']
```

```
In [ ]: # Define a dictionary explicitly
```

```
age = {'Amy':20, 'Ben':40, 'Chris':35}
age
```

```
In [ ]: # different types

age = {'Amy':20, 'Ben':40, 'Chris':35, 100:'old'}
age
```

```
In [ ]: # no duplicated keys

age = {'Amy':20, 'Ben':40, 'Chris':35, 'Chris':45}
age
```

```
In [ ]: # change the value of the specified key (mutable)

age['Amy'] += 22
# Equivalent to: age['Amy'] = age['Amy'] + 22
age
```

```
In [ ]: # if the key is not in the dictionary,
# the new key:value pair will be added

age['David'] = 28
age
```

```
In [ ]: # The `in` operator checks whether a key is present within the dictionary

print('Amy' in age)
print('Emma' in age)
```

```
In [ ]: # The `del` command removes the specified key:value pair.

del age['Amy']
age
```

Some Dictionary Methods

- `items()`
 - return iterable view of (key, value) tuples
- `keys()`
 - return view of the dictionary's keys
- `values()`
 - return view the dictionary's values.
- `copy()`
 - create a new dictionary with a copy of the references from the original dictionary.
- `update()`
 - update the dictionary with the elements from another dictionary.

More: https://www.w3schools.com/python/python_ref_dictionary.asp

Future: `frozendict` , <https://peps.python.org/pep-0814>

```
In [ ]: age.keys()
```

```
In [ ]: age.values()
```

```
In [ ]: d1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30_000}
        d2 = {'Thirty': 30, 'Fourty': 40, 'Fifty': 50}

        d3 = d1.copy()
        d3.update(d2)
        print(d3)
```

```
In [ ]: d1
```

```
In [ ]: # A "copy", not a .copy()!

        d4 = d1
        d4
```

```
In [ ]: d1['Ten'] = 10_000
        d1
```

```
In [ ]: d4
```

```
In [ ]: d4['Twenty'] = 123456789
        d1
```

```
In [ ]: # A dictionary view is a lightweight, **dynamic** wrapper
        # around the dictionary's contents.
        view = age.items()
        view
```

```
In [ ]: age['new'] = 75
        view
```

Views are iterable, support membership testing, are not indexable, and do not copy the data. Some are "set-like", if unique.

Sets

Python also supports a container data type that behaves like a mathematical **set**. Sets are:

- Unordered (**NOT INDEXED**)
- Only contain, **at most**, one copy of each object added to them.
- Sets can only contain immutable values (**cannot be nested**)
 - integers
 - strings
 - tuples

Define a set by a pair of curly braces `{}` .

To define an **empty** set:

```
s = set()
```

```
In [ ]: # Define a populated set
```

```
st = {'a', 'b', 'a', 1, 2, 1}
st
```

```
In [ ]: # Convert a list to a set to obtain the unique items in the list.
```

```
list_of_grades = ['A', 'B', 'A', 'C', 'B', 'A']
unique_grades = set(list_of_grades)
unique_grades
```

A `frozenset()` is the **immutable** version of a `set`.

- They will not work with `.add`, `.remove`, etc.
- With immutability, they become **hashable** (i.e., value is mapped to an integer), meaning that they could be used as the `key` of a dictionary.

Set Operations

- `len()` - returns the number of items.
- `in` - membership test
- Union: `|` or `a.union(b)`
 - Returns a set that contains **all separately unique items** from both sets `a` and `b` (without duplicates)
- Intersection: `&` or `a.intersection(b)`
 - Returns a set that contains the items that are **common** to sets `a` and `b`
- Difference `-` or `a.difference(b)`
 - Returns a set that contains the items that only exist in set `a`, not in set `b`

```
In [ ]: a = {1, 3, 7, 11}
b = {3, 5, 7, 9}

print(f'Union: {a | b}')
print(f'Intersection: {a & b}')
print(f'Difference: {a - b}')
print(f'Difference: {b.difference(a)}')
```

Some Set Methods

- `add()`
 - add one item to the set
- `update()`
 - add multiple items to the set
- `discard()`
 - remove one specified item from the set
- `clear()`
 - remove all items from the set

```
In [ ]: grades = {'A', 'B', 'C'}  
grades.add('D')  
grades
```

```
In [ ]: grades.update(['F', 'I'])  
grades
```

```
In [ ]: grades.discard('I')  
grades
```

```
In [ ]: grades.clear()  
grades
```

Type Casting

- **Type casting** is the action of *changing the type* of an object.

Examples:

```
In [ ]: # Convert a float to int  
  
int(1.8)
```

```
In [ ]: # Convert a string to int  
  
int('123')
```

```
In [ ]: # Convert a boolean to int  
  
int(True)
```

```
In [ ]: # Convert an int to float  
  
float(1)
```

```
In [ ]: # Convert a string to float  
  
float('12.3')
```

```
In [ ]: # Convert a list to a set  
  
some_grades = ['A', 'B', 'A', 'C', 'A', 'B']  
set(some_grades)
```

```
In [ ]: # Convert a set to a list  
  
some_grades = {'A', 'B', 'A', 'C', 'A', 'B'}  
list(some_grades)
```

Flow Control

- `if-elif-else` statements
- `for` loops

- `while` loops
- `break`, `continue`, `pass`, `loop` `else`

Syntax:

- a header line followed by a colon `:`, then an **indented** block of text.

Unlike C or Java: no parentheses and no semicolons are required.

if

if *condition*:

statement(s)

- This evaluates the *condition* and will execute *statement(s)* only **if** the *condition* is `True`.

```
In [ ]: # Test different grade values to understand how the if statement works
grade = 30

if grade < 40:
    print('Fail')
```

- The **body** of the `if` statement is indicated by the **indentation**.

```
In [ ]: # Test different grade values to understand how the if statement works
grade = 50

if grade < 40:
    print('Fail')
    print('You need to re-exam.')
print('This is always printed.')
```

- Several conditions can be combined using **logical operators**.

```
In [ ]: grade = 50

if grade < 40 and grade > 0:
    print('Fail')
```

if-else

if *condition*:

if-part-statement(s)

else:

else-part-statement(s)

- This evaluates the *condition* and will execute *if-part-statement(s)* if the *condition* is `True` , otherwise the *else-part-statement(s)* will be executed.

```
In [ ]: # Test different grade values to understand how the if statement works
grade = 48

if grade < 40:
    print('Fail')
else:
    print('Pass')
```

elif

```
if condition1:

    block1-statement(s)

elif condition2:

    block2-statement(s)

else:

    else-block-statement(s)
```

- This evaluates *condition1* and will execute *block1-statement(s)* if *condition1* is `True` .
- If *condition1* is `False` , the condition of the next `elif` block is checked and so on.
- If all the conditions are `False` , the *else-block-statement(s)* will be executed.
- Note: **only one** block among the several `if...elif...else` blocks will be executed.
- It can have multiple `elif` blocks, but **only one** `else` block.

```
In [ ]: grade = 43

if grade < 40:
    print('Fail')
elif grade < 50:
    print('Threshold Pass')
else:
    print('Third Class or above')
```

Nested if

- We can have an `if...elif...else` statement inside another `if...elif...else` statement.
- Any number of these statements can be nested inside one another.
- Indentation is the **only way** to determine the level of nesting.

```
In [ ]: # try different grade values to understand how the if statement works
grade = 150

if grade >= 0 and grade <= 100:
    if grade < 40:
        print('Fail')
    elif grade < 50:
        print('Threshold Pass')
    else:
        print('Third Class or above')
else:
    print('Invalid grade')
```

for loops

for *x* **in** *iterable*:

loop-body

- *x* is the variable that **takes the value** of the item inside the *iterable* on each iteration.
 - *x* is **NOT a counter**.
- Some iterable *sequence* types: strings, lists, tuples, range objects.
 - These are ordered.
- Other iterable types: sets, dictionary keys
 - **THINK ABOUT ORDERING!**

Iterable objects: https://www.pythonlikeyoumeanit.com/Module2_EssentialsOfPython/Iterables.html

```
In [ ]: # for loop with strings:
str1 = 'Python'
for x in str1:
    print(x)
```

for loops over a range()

- Quickly generate a sequence of integers.
- range(*start*, *N*, *step*) generates an **iterable** object starting at *start*, ending at *N*-1, with a *step*
- range(*N*) generates an **iterable** starting at 0 and ending at *N* - 1
- range(*N*) **does not include N** !

More: <https://pynative.com/python-range-function/>

```
In [ ]: for i in range(8):
        print(i)
```

```
In [ ]: for i in range(1, 10, 2):
        print(i)
```

```
In [ ]: for i in range(10, 0, -1):
        print(i)
```

for loops over iterable objects: Lists, Tuples, Sets, Dictionaries

```
In [ ]: # for Loops with Lists - values:

xs = [1, 4, 2, 3]

for x in xs:
    print(x)
```

Use `enumerate(var)` to get **two** loop variables:

- the *count* of the current iteration, and
- the *value* of the item at the current iteration.

The `enumerate()` function takes a collection (e.g., a tuple) and returns it as an enumerate **object**.

The `enumerate()` function adds a counter as the key of the enumerate object.

```
In [ ]: # for Loops with Lists - index and values:

xs = [1, 4, 2, 3]

for i, v in enumerate(xs):
    print(i, v)
```

```
In [ ]: # for Loops with Tuples

t = (1, "two", [3, 4, 5])

for x in t:
    print(x)
```

```
In [ ]: # for Loops with Sets

s = {1, 2, 2, 3, "Hello", 6, 'm', (3, 3)}

for x in s:
    print(x)
```

```
In [ ]: # for Loops with Dictionaries - keys:

d = {1: "Hello", 2: "world"}

for k in d:
    print(k)
```

```
In [ ]: # for Loops in Dictionaries - keys and values:

d = {1: "Hello", 2: "world"}

for k, v in d.items():
    print(k, v)
```

```
In [ ]: d.items() # Like a list of tuples
```

while loops

while *condition*:

loop-body

- `while` loops execute a block of code as long as a condition is `True`.
- Remember to **update some component of the condition within the loop** so that the condition may become `False` at some point.

```
In [ ]: counter = 5

while counter > 0:
    print("Hello world")
    counter -= 1
```

```
In [ ]: x = y = 0.0

steps = 0

while (x * x + y * y) < 10.0 and steps < 10:
    xx = x * x - y * y + 0.1
    y = 2.0 * x * y + 0.02
    x = xx
    steps = steps + 1
    print(steps, x, y)
```

break

Terminate the current `for` / `while` loop by using the `break` statement to exit the code block **immediately**.

```
In [ ]: for x in "Python":
        if x == "h":
            break
        print(x)

print("The end")
```

continue

Skip the **remainder** of the current iteration of a `for` / `while` loop and move to the next iteration.

```
In [ ]: for x in "Python":
        if x == "h":
            continue
        print(x)

print("The end")
```

pass

Do nothing. **Ignore** the condition in which it occurred and proceed to run the program as usual.

Often, this is a **placeholder** in code under development, noting that something might be done for the condition in a future revision. It could also be used as **minimal exception handling**.

```
In [ ]: for x in "Python":
        if x == "h":
            pass
        print(x)

print("The end")
```

Loop else

The `else` block of a `for / while` loop is **only reached if the loop ends without a break statement** being encountered (if it exits normally).

```
In [ ]: # example: check if num is a prime number
num = 17

for n in range(2, num):
    print("tested", n)
    if num % n == 0:
        print(num, "is divisible by", n)
        break
else:
    print(num, "is prime")
```

match-case

The `match` (Switch/Case) statement in Python 3.10+

Python 3.10 introduced a new feature called **structural pattern matching**, often used with what is informally referred to as a `switch/case` statement.

- Structural patterns are used to check whether the *shape and contents* of an object fit a *pattern*.
- These enable selecting between *code paths* based on the form and contents of a value, in a manner similar to `if-elif-else`, but more concisely

This addition lets you replace multiple `if / elif` checks with a cleaner, more readable syntax. This enhances:

- **Readability**: often clearer than a long chain of `if / elif / else`.
- **Maintainability**: adding and rearranging cases often becomes simpler.
- **Extensibility**: enables matching against complex patterns, not just single values.
 - includes sequences, mappings, and classes
 - **See practical example.**

`match` *subject*:

```
case pattern1:
    case-body

case pattern2:
    case-body

case _:
    default-case-body
```

For a presented *subject*, check whether whether there is a match to one of the coded *patterns*.

Pattern matching can be simple, as above, or much more complex, for instance, testing on a collection of values, dictionary keys, or class attributes: <https://peps.python.org/pep-0636/>.

The following example takes an **HTTP status code** (e.g., 200, 404, 500) and returns a suitable description. You might see a similar pattern in web services or APIs that handle different server responses.

```
In [ ]: test_codes = [200, 400, 404, 500, 999, "blip"]

for code in test_codes:
    match code:
        case 200:
            print(f"{code} - OK: The request has succeeded.")
        case 400:
            print(f"{code} - Bad Request: The server could not understand the request.")
        case 404:
            print(f"{code} - Not Found: The requested resource could not be found.")
        case 500:
            print(f"{code} - Internal Server Error: The server encountered an error.")
        case _:
            # The underscore is a wildcard and catches any case not listed above
            print(f"{code} - Unknown status code.")
```

We can also test **multiple patterns** for a specific case.

```
In [ ]: test_codes = [200, 400, 404, 500, 999, "blip"]
for code in test_codes:
    match code:
        case 200:
            print(f"{code} - OK: The request has succeeded.")
        case 400 | 404 | 500:
            print(f"{code} - Something unpleasant happened.")
        case 404:
            print(f"{code} - Not Found: The requested resource could not be found.")
        case 500:
            print(f"{code} - Internal Server Error: The server encountered an error.")
        case _:
            # The underscore is a wildcard and catches any case not listed above
            print(f"{code} - Unknown status code.")
```

We can condition on `type`, and the order of case statements matters.

```
In [ ]: test_codes = [200, 400, 404, 500, 999, 'blip']
for code in test_codes:
    match code:
        case 200:
            print(f"{code} - OK: The request has succeeded.")
        case 400 | 404 | 500:
            print(f"{code} - Something unpleasant happened.")
        case 404:
            print(f"{code} - Not Found: The requested resource could not be found.")
        case 500:
            print(f"{code} - Internal Server Error: The server encountered an error.")
        case str():
            print(f'{code = } This is a string.')
        case int():
            print(f'{code=} This is an int.')
        case _:
            # The underscore is a wildcard and catches any case not listed above
            print(f"{code} - Unknown status code.")
```

```
In [ ]: # Let's explore the use of more complex **generic** patterns
```

```
def describe(value):
    match value:
        case [x, y]: # matches any sequence of length 2, the brackets are not important
            return f"I found a pair of entries: {x}, {y}."
        case {"name": n, "age": a}: # matches a mapping with those keys
            return f"I found a dictionary entry with 'name' and 'age' keys: name {n}, age {a}."
        case Point(m, n): # Checks whether the value is a valid instance.
            # If so, we can access the passed arguments as these local names,
            # but their values are actually determined by a check of the
            # class or instance attribute names in the order defined by the
            # __match_args__ class attribute. It's a messy construct!
            return f"I found a Point instance at ({m}, {n})."
        case _:
            return "Something else"

class Point:
    __match_args__ = ("x", "y")
    y = 100
    def __init__(self, x, y):
        self.x, self.y = x, y

# Example usage
print(describe( (1, 2) ))
print(describe( {"name": "Alice", "age": 30} ))
print(describe( Point(1, -1) ))

# Test again, changing self.y to be defined as self.z
# or by changing the order of x and y in the __init__.
```

Caution: Inside a `case` pattern, **names are bindings by default, not references to existing variables.**

- That is, if `x` or `y` had already been defined they would be considered as new generic names for the construction of the pattern **within the `case` statements.**

Comprehensions

Comprehensions allow construction of collections (e.g., `list`, `set`, `dict`) via iterative processing without the need to write loops and construction infrastructure.

- Short and concise!

For a `list`, these are composed of a set of **square brackets** containing an expression, followed by a `for` clause, then zero or more `for` or `if` clauses:

```
variable = [ expression for var in collection <if condition> ]
```

```
In [ ]: # First, with a Loop and a List

# Construct a List that contains the cube of the values
# from 1 to 20 that are divisible by 5.

result = []

for v in range(1, 20):
    cube = v**3
    if cube % 5 == 0:
        result.append(cube)

result
```

```
In [ ]: # Now, with List comprehension
[v**3 for v in range(1, 20) if v**3 % 5 == 0]
```

```
In [ ]: # Now, with List comprehension and the walrus operator
[cubed for v in range(1, 20) if (cubed := v**3) % 5 == 0]
```

```
In [ ]: # For sets:
{v**3 for v in range(1, 20) if v**3 % 5 == 0}
```

```
In [ ]: # For dictionaries (with value as the key):
{v:v**3 for v in range(1, 20) if v**3 % 5 == 0}
```

```
In [ ]: # Nested comprehension
[(x, y) for x in range(1, 5) for y in range(1, 5)]
```

```
In [ ]: # Equivalent nested Loops

mylist=[]
for x in range(1, 5):
    for y in range(1, 5):
        mylist.append((x, y))
mylist
```

Note: be particularly careful about including function calls in comprehensions. The use of those with "side-effects" can muddle the intent of the comprehension expression, and return types may not always be obvious or well-suited to collection construction.

The Walrus Operator

A neat piece of **syntactic sugar** introduced in *Python 3.8*.

`:=` is an **assignment expression** that assigns values to variables as part of larger expressions.

- can make code more concise by combining the computation and assignment in one step
- especially handy in loops, comprehensions, and conditional statements

Traditionally, one would need to compute a value on one line and then use it in a condition on the next. With the walrus operator, we can **both compute the value and assign it** to a variable within the same expression.

The operator looks like `:=` (its shape often reminds people of a walrus, hence the nickname). It takes the value on its right-hand side, **assigns** it to the variable on its left-hand side, and then **returns** that value.

Using the Walrus Operator in a `while` Loop

When reading input until a blank line is entered, use the walrus operator to both read and test the input in one line:

```
In [ ]: '''
Here, line := input(...) reads user input and assigns it to line at the same time.
The loop continues until an empty string is entered.
'''

while (line := input("Enter something (or leave blank to quit): ")) != "":
    print(f"You entered: {line}") # Access the newly assigned variable.

print("Goodbye!")
```

Simplifying Conditional Checks

Use the walrus operator in an `if` statement to capture a value and then use it immediately:

```
In [ ]: '''
Instead of calling len(nums) twice, the walrus operator assigns the length
to n and simultaneously checks if it is greater than 3.
At the same time, we assign and return 3 to thresh.
'''

nums = [1, 2, 3, 4, 5]

if (n := len(nums)) > (thresh := 3):
    print(f"The list has {n=} elements, which is more than {thresh=}.")
```

Combining Assignment in a List Comprehension

The walrus operator can also streamline **list comprehensions** when we want to both **compute and filter** based on a value:

```
In [ ]: '''
For each number n in nums, the expression (square := n * n)
computes and assigns its square to square.
The comprehension then includes only those square values that exceed 10.
'''
```

```
nums = [1, 2, 3, 4, 5]
```

```
# Compute the square and filter only those squares that are greater than 10.
```

```
squares = [square for n in nums if (square := n * n) > 10]
```

```
print(squares)
```

Without the walrus operator, the above operation cannot be *efficiently* performed with a comprehension. Instead, we would need to rely on a loop, computing the square once per iteration:

```
result = []
```

```
for n in nums:
```

```
    square = n * n
```

```
    if square > 10:
```

```
        result.append(square)
```

Notes on Walrus Operator Utility

Conciseness and Readability

- By **reducing repetition** (e.g., avoiding multiple calls to the same function), the walrus operator can simplify code and reduce the risk of mistakes.

Performance

- Where computing a value is expensive (such as reading from a file or making a network request), using the walrus operator prevents the need for **redundant computations**.

Use Cases

- *Repeated overwriting assignments with reuse.*
- Loops: Read and process data until a termination condition is met.
- Conditionals: Assign a value while testing it in if statements.
- Comprehensions: Avoid repeating calculations inside filters.

Caution

- Overusing the walrus operator or using it in very complex expressions can sometimes **harm readability** for others. It is best used when it genuinely reduces redundancy and clarifies the code.
- Sometimes, it can be just as effective, if not clearer, to simply store a value in a variable via common assignment, particularly if the value is unlikely to change frequently.

Summary and Independent Study Resources

- Python offers a **broad set of data type options** that can be implemented depending upon the requirements of the tasks to perform.
- Data types differ in being **mutable**, **ordered**, or **nestable**.
- **if...elif...else** statements are used for decision making.
- **for** loops are used to iterate over a Sequence or other Iterable objects (range objects, Strings, Lists, Tuples, Sets, Dictionaries).

- **while** loops are used in general when we don't know the number of times to iterate beforehand.
 - **break**, **continue**, and **pass** statements can alter the flow of a normal loop.
 - **match...case** structures provide a more intuitive way to handle multiple conditions compared to traditional if/elif/else blocks.
 - **Comprehensions** allow you to build a **collection** (list, set, dict) by specifying how it should be constructed in a **single line**.
 - **The Walrus Operator** combines assignment with evaluation in a single expression, reducing code redundancy and potentially simplifying loops and conditional checks by assigning values on the fly.
-

Online Tutorials:

- if-elif-else
 - <https://pynative.com/python-control-flow-statements/>
 - for loops
 - <https://pynative.com/python-for-loop/>
 - while loops
 - <https://pynative.com/python-while-loop/>
 - break, continue, and pass
 - <https://pynative.com/python-break-continue-pass/>
 - Nested loops
 - <https://pynative.com/python-nested-loops/>
 - Sequences
 - <https://techvidvan.com/tutorials/python-sequences>
 - Iterables
 - https://www.pythonlikeyoumeanit.com/Module2_EssentialsOfPython/Iterables.html
 - range()
 - <https://pynative.com/python-range-function/>
 - match-case
 - <https://learnpython.com/blog/python-match-case-statement/>
 - Comprehensions
 - <https://www.programiz.com/python-programming/list-comprehension>
 - Assignment Expressions (walrus operator)
 - <https://www.digitalocean.com/community/tutorials/how-to-use-assignment-expressions-in-python>
-

Official Python Documentation:

- <https://docs.python.org/3/tutorial/controlflow.html>

Study References:

- Lutz, M. (2025). Learning Python: Powerful Object-Oriented Programming
 - **Parts II & III (Chapters 4-14)**
 - McKinney (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython.
 - **Chapters 2 and 3.1**
-

More Exercises:

- <https://pynative.com/python-if-else-and-for-loop-exercise-with-solutions/>

Critical information regarding assignment and copying variables:

- <https://realpython.com/copying-python-objects/>
-

In This Week's Practical

Explore implementation of the above tools to solve problems.

Next Week

Functions, Classes, and Modules

Suggested Reading:

- Lutz, M. (2025). Learning Python: Powerful Object-Oriented Programming
 - **Parts IV, V, and VI**
- McKinney (2022). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython.
 - **Chapter 3**